

Probabilistic Models

Introduction to N-Gram Models

- An **N-Gram model** predicts words based on the previous N words.
- Used in speech recognition, autocomplete, translation, and spell checking.
- **Larger N = more context, but higher complexity.**
- Formula:
 - Unigram ($n = 1$): $P(w_n)$ #只看單字在語料庫中的出現頻率
 - Bigram ($n = 2$): $P(w_n | w_{n-1})$
 - Trigram ($n = 3$): $P(w_n | w_{n-1}, w_{n-2})$
- Example: If past sentences include:
 - “I love NLP” (40%)
 - “I love AI” (30%)
 - “I love programming” (30%)
 - **Then $P(\text{NLP} | \text{I love}) = 0.4$**

Where Are N-Gram Models Used?

- Speech Recognition → Predicts missing words.
- Autocomplete & Spell Checking → Suggests next words.
- Machine Translation → Improves word prediction accuracy.
- Text Generation → Creates realistic sentences.

When Calculating the Probability of a Sequence...

Sequence Definition : $\{x_1, x_2, \dots, x_T\}$

Joint Probability (Goal) : $p(x_1, x_2, \dots, x_T)$

The Chain Rule of Probability :

$$p(x_1, x_2, \dots, x_T) = p(x_1) \cdot p(x_2 | x_1) \cdot p(x_3 | x_1, x_2) \cdot \dots \cdot p(x_T | x_{T-1}, \dots, x_1)$$

or using product notation :

$$p(x_1, x_2, \dots, x_T) = p(x_1) \prod_{t=2}^T p(x_t | x_{t-1}, x_{t-2}, \dots, x_1)$$

Key Computational Problem :

$$p(x_t | x_{t-1}, x_{t-2}, \dots, x_1)$$

When Calculating the Probability of a Sequence...

Sequence Definition : $\{x_1, x_2, \dots, x_T\}$

Joint Probability (Goal) : $p(x_1, x_2, \dots, x_T)$

The Chain Rule of Probability :

$$p(x_1, x_2, \dots, x_T) = p(x_1) \cdot p(x_2 | x_1) \cdot p(x_3 | x_1, x_2) \cdot \dots \cdot p(x_T | x_{T-1}, \dots, x_1)$$

or using product notation :

$$p(x_1, x_2, \dots, x_T) = p(x_1) \prod_{t=2}^T p(x_t | x_{t-1}, x_{t-2}, \dots, x_1)$$

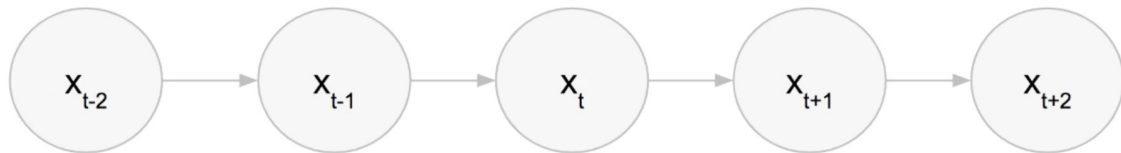
Key Computational Problem :

$$p(x_t | x_{t-1}, x_{t-2}, \dots, x_1)$$

← Calculating this probability can be a nightmare when T becomes large due to data sparsity

The Markov Property

- The Markov property is a very restrictive assumption on the dependency structure of the **joint probability**
- The probability of transitioning to the next state depends only on the current state, not past states



- This implies that the joint probability takes the form:

$$p(x_1, \dots, x_T) = p(x_1) \prod_{t=2}^T p(x_t \mid x_{t-1})$$

The Markov Property

What the Markov property says

$$p(x_t \mid x_{t-1}, x_{t-2}, \dots) = p(x_t \mid x_{t-1})$$

What the Markov property also says

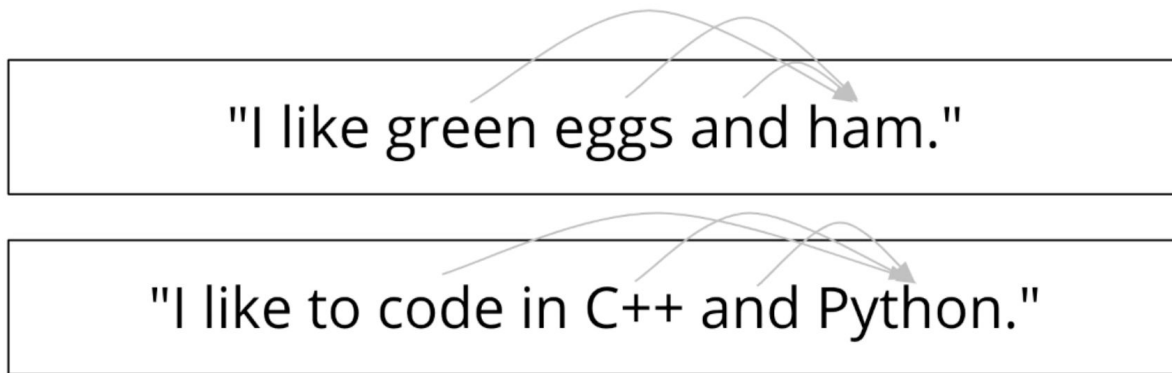
x_t is independent of $x_{t-2}, x_{t-3}, \dots$

Why is the Markov Property Useful?

- Consider a Markov model of the English language
- Suppose we choose 2000 of the most common words
- Suppose we want to get the probability distribution for the 10th word in a sentence given the previous nine words
- Our model is $p(x_{10} \mid x_9, x_8, \dots, x_1)$
- Each “x” has 2000 possible values (words)
- How many probabilities to estimate? $2000 \times 2000 \times 2000 \dots = 2000^{10}$
- Q: Do we have a sufficient amount of data?

The Markov “Asumption”

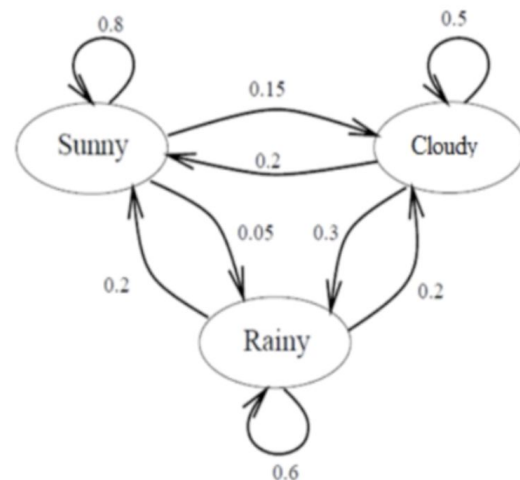
- We assume the Markov property holds, even when it does not



- Do “ham” and “Python” only depend on “and”? No!
- “All models are wrong, but some are useful” - George Box

The Markov Model

- In this section, we'll consider sequences of **categorical symbols**, which we'll refer to as “states”
- Example, weather: {sunny, rainy, cloudy}
- Example, POS tags: {noun, verb, adjective}



Notation

- We'll use **s** for “state”

$$s(t) = s_t = \textit{state at time } t$$

- Time will also be discrete ($t = 1, 2, \dots$) (e.g., no such thing as $t = 1.5$)
- We'll **number** the states from $1, 2, \dots, M$
- M = total number of possible states (e.g., $M = 3$ for sunny/rainy/cloudy)
- We use i or j to **index** the state space

$p(s_t = i)$ means : *probability that state at time t is 'i'*

State Distribution

- We have $p(s_t = 1)$, $p(s_t = 2)$, ..., $p(s_t = M)$
- This is M probability values - together they form a **distribution**
- Ex: “What is the probability that it will be rainy on Sunday?”

Ans: $p(s_{\text{sunday}} = \text{rainy})$

$p(s_t)$ = *state distribution (length M vector)*

State Transitions

$$p(s_t = j \mid s_{t-1} = i)$$

- “Probability that state at time t is j , given that the state at time $t-1$ was i ”
- How many of these probability values exist? (for all i and j)
- Since both i and j can take any value from $1 \dots M$, there are M^2 values

State Transition Matrix

$$A_{ij} = p(s_t = j \mid s_{t-1} = i), \quad \forall i = 1 \dots M, j = 1 \dots M$$

- A is an MxM matrix
- Convention: first index (row) = previous state, second index (column) = next state

State Transition Matrix

But where is t ?

$$A_{ij} = p(s_t = j \mid s_{t-1} = i), \quad \forall i = 1 \dots M, j = 1 \dots M$$

- In general, we could have $A_{ij}(t)$
- When A doesn't depend on t : **time-homogeneous Markov process**
- Note: Natural language is NOT strictly time-homogeneous, but here we drop t to prevent parameter explosion and solve the data sparsity issue

Initial State

What's the probability
of the first word?

The quick brown fox jumps over the lazy dog.

- To quantify the probability of the first state in a sequence, we use the **initial state distribution**

$$\pi_i = p(s_1 = i) \quad (\text{for } i = 1 \dots M)$$

Recap

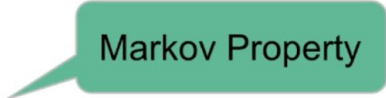
$$A_{ij} = p(s_t = j \mid s_{t-1} = i)$$

$$\pi_i = p(s_1 = i)$$

- Given A and π , and a sequence $\{s_1, s_2, \dots, s_t\}$, what is a probability of seeing that sequence?
- How do we find A and π given a dataset?

Probability of a Sequence

$$p(s_{1...T}) = p(s_1)p(s_2|s_1)p(s_3|s_1, s_2)\dots$$

$$p(s_{1...T}) = p(s_1) \prod_{t=2}^T p(s_t|s_{t-1})$$


Markov Property

$$p(s_{1...T}) = \pi_{s_1} \prod_{t=2}^T A_{s_{t-1}, s_t}$$

Start thinking about how you'd implement this in Python code

Training a Markov Model

- Suppose we flip a coin a bunch of times - how do we estimate $p(\text{heads})$?

$$p(\text{heads}) \approx \frac{\text{count}(H)}{\text{total tosses}}$$

- That's just the binary case. What if we have $M > 2$?

$$p(\text{"cat"}) \approx \frac{\text{count}(\text{"cat"})}{\text{total word count}}$$

Estimating A and π (Training)

N = number of
sequences in dataset

Note: the 'hat'
means 'estimate'

$$\hat{\pi}_i = \frac{\text{count}(s_1 = i)}{N}$$

$$\hat{A}_{ij} = \frac{\text{count}(i \rightarrow j)}{\text{count}(i)}$$

Example:
 $\text{count}(\text{"the cat"}) / \text{count}(\text{"the"})$

Probability of a Sequence

$$p(s_{1...T}) = \pi_{s_1} \prod_{t=2}^T A_{s_{t-1}, s_t}$$

- Only involves multiplication
- What if one of the value is 0? Transition never appears in training set
- The result becomes 0!

Add-One Smoothing

- Give a small probability to every possible transition
- Add a “fake count” of 1 for each (i, j) transition

$$\hat{A}_{ij} = \frac{\textit{count}(i \rightarrow j) + 1}{\textit{count}(i) + M}$$

Adding M to the denominator ensures
that each row of A sums to 1

Add-One Smoothing

- We can do this for the initial state distribution too

$$\hat{\pi}_i = \frac{\textit{count}(s_1 = i) + 1}{N + M}$$

Add-Epsilon Smoothing

- We can make it more smooth ($\varepsilon > 1$) or less smooth ($\varepsilon < 1$)

$$\hat{A}_{ij} = \frac{\textit{count}(i \rightarrow j) + \varepsilon}{\textit{count}(i) + \varepsilon M}$$

$$\hat{\pi}_i = \frac{\textit{count}(s_1 = i) + \varepsilon}{N + \varepsilon M}$$

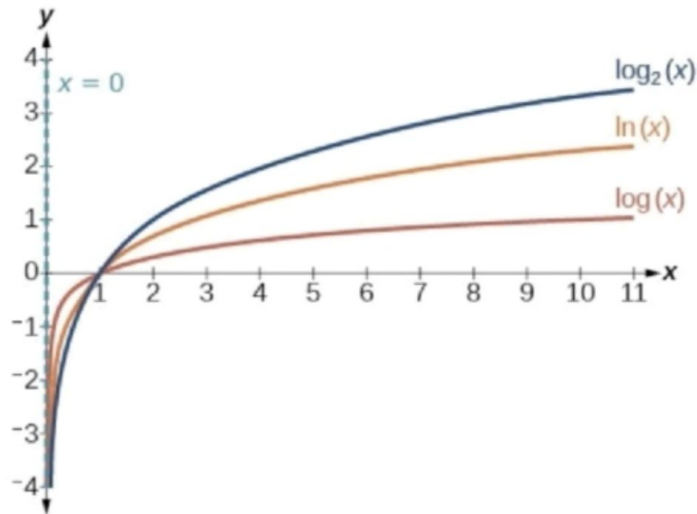
Computing the Probability of a Sequence

$$p(s_{1...T}) = \pi_{s_1} \prod_{t=2}^T A_{s_{t-1}, s_t}$$

- This involves multiplying many small numbers together
- Common to use 20k - 50k vocabulary size in English
- As you multiply small numbers together, they approach 0!
- Computers don't have infinite precision; eventually it just rounds to 0
- Problematic: what if we want to compare 2 sequences?

Working with Log Probabilities

- Solution: compute log probabilities instead
- We don't need the actual probability value, since what we usually want to do is compare (e.g., is one sequence more likely than another?)

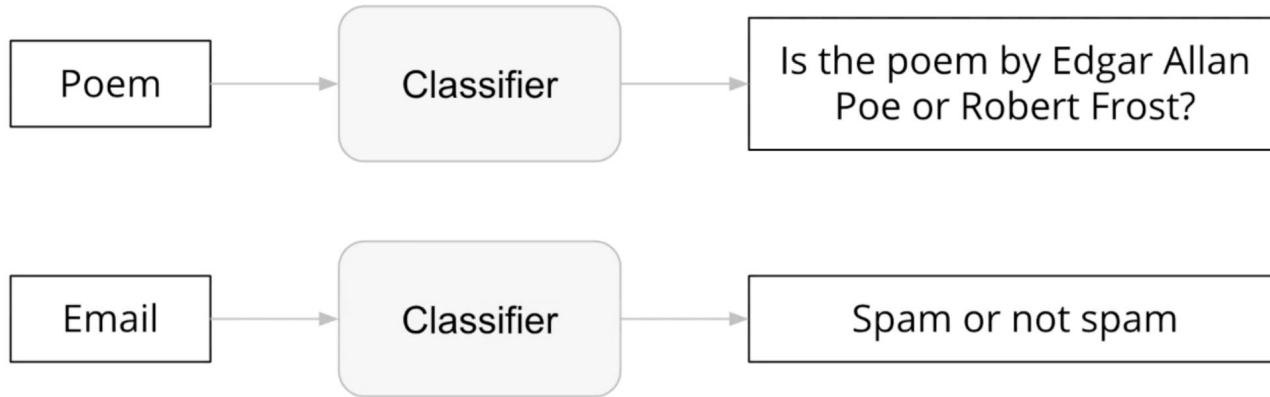


Working with Log Probabilities

$$\log p(s_{1..T}) = \log \pi_{s_1} + \sum_{t=2}^T \log A_{s_{t-1}, s_t}$$

- Since $\log(AB) = \log(A) + \log(B)$
- Note: do not compute the product and then take the log (to avoid $\log(0)$)

Build a Text Classifier Using Markov Models

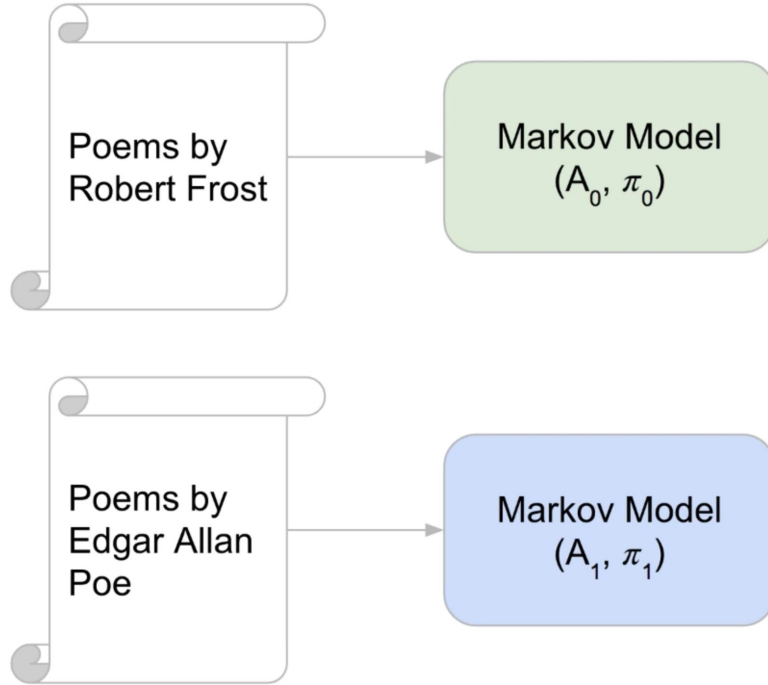


Supervised or Unsupervised?

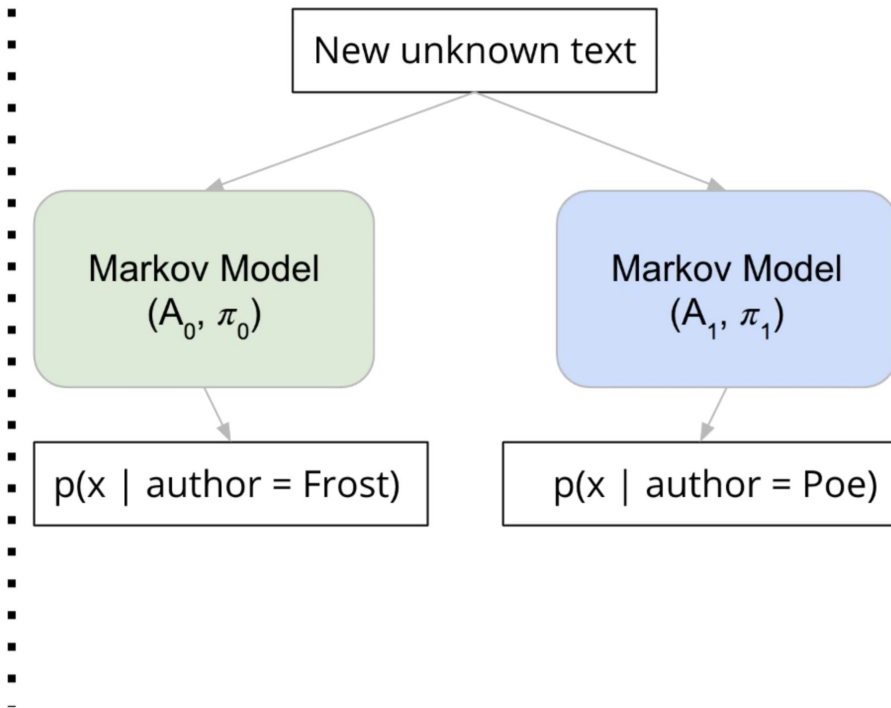
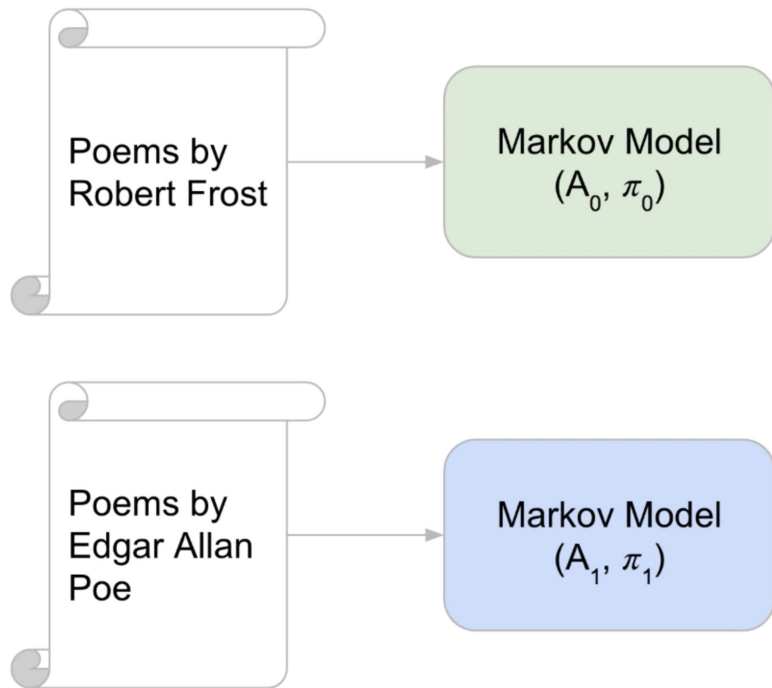
- Text classification is an example of supervised learning, but Markov models are unsupervised (training data is just text; no labels)
- Answer: We must apply Bayes' rule (and build a **Bayes classifier**)

$$p(y \mid x) = \frac{p(x \mid y)p(y)}{p(x)}$$

Bayes Classifier



Bayes Classifier



Maximum A Posteriori (MAP)

- We have $p(\text{poem} \mid \text{author})$, but we want $p(\text{author} \mid \text{poem})$ (a **probability distribution**)
- Then we can apply the following **decision rule**:

If it's not obvious,
"poem" = x , and
"author" = class

$$k^* = \arg \max_k p(\text{class} = k \mid x)$$

Apply Bayse Rule (In this example)

$$p(author \mid poem) = \frac{p(poem \mid author) p(author)}{p(poem)}$$

How to Find the Probability Distribution

Posterior

Likelihood

Prior

$$p(author \mid poem) = \frac{p(poem \mid author) p(author)}{p(poem)}$$

Simplifying the Decision Rule

$$k^* = \arg \max_k \frac{p(\textit{poem} \mid \textit{author} = k) p(\textit{author} = k)}{p(\textit{poem})}$$

$$k^* = \arg \max_k p(\textit{poem} \mid \textit{author} = k) p(\textit{author} = k)$$

$$k^* = \arg \max_k \log p(\textit{poem} \mid \textit{author} = k) + \log p(\textit{author} = k)$$

Simplifying the Decision Rule

$$k^* = \arg \max_k \frac{p(\textit{poem} \mid \textit{author} = k) p(\textit{author} = k)}{p(\textit{poem})}$$

$p(\textit{poem})$ can be ignored

$$k^* = \arg \max_k p(\textit{poem} \mid \textit{author} = k) p(\textit{author} = k)$$

$$k^* = \arg \max_k \log p(\textit{poem} \mid \textit{author} = k) + \log p(\textit{author} = k)$$

Simplifying the Decision Rule

$$k^* = \arg \max_k \frac{p(\textit{poem} \mid \textit{author} = k) p(\textit{author} = k)}{p(\textit{poem})}$$

Apply log function

$$k^* = \arg \max_k p(\textit{poem} \mid \textit{author} = k) p(\textit{author} = k)$$

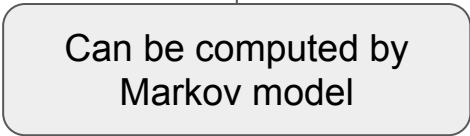
$$k^* = \arg \max_k \log p(\textit{poem} \mid \textit{author} = k) + \log p(\textit{author} = k)$$

Simplifying the Decision Rule

$$k^* = \arg \max_k \frac{p(\textit{poem} \mid \textit{author} = k) p(\textit{author} = k)}{p(\textit{poem})}$$

$$k^* = \arg \max_k p(\textit{poem} \mid \textit{author} = k) p(\textit{author} = k)$$

$$k^* = \arg \max_k \log p(\textit{poem} \mid \textit{author} = k) + \log p(\textit{author} = k)$$



Can be computed by
Markov model

Maximum Likelihood

- We can simply take the argmax of the likelihood if the **prior is uniform** (i.e. we have no reason to believe, given no other info, that one author is more likely than another)

$$k^* = \arg \max_k \log p(\textit{poem} \mid \textit{author} = k)$$

If $p(\textit{author})$ is uniform

Recap

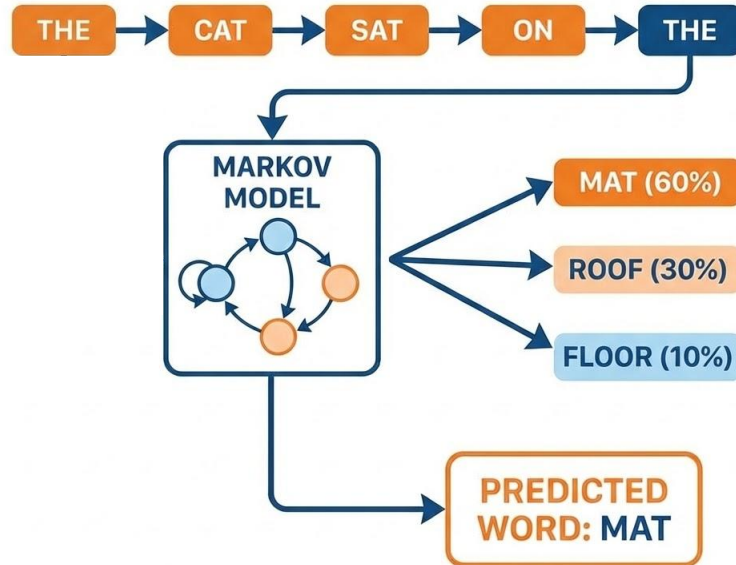
- We train a separate Markov model for each class
- Each model gives us $p(x \mid \text{class} = k)$ for all k
- General form of decision (MAP): $k^* = \operatorname{argmax}_k p(\text{class} = k \mid x)$
- Posterior can be simplified since we don't need its actual value
- Simplified MAP: $k^* = \operatorname{argmax}_k \log p(x \mid \text{class} = k) + \log p(\text{class} = k)$
- Maximum likelihood: $k^* = \operatorname{argmax}_k \log p(x \mid \text{class} = k)$

Text Classifier Exercise Prompt

- You'll be given poems by 2 authors: Edgar Allan Poe and Robert Frost
- Build a classifier that can distinguish between the 2 authors
- Compute train and test accuracy
- Check for class imbalance, compute F1-score if imbalanced

Using Markov Models to Generate Text

- Classifying text: supervised learning (we have labels)
- Generating text: unsupervised learning (no labels)



Problem with Markov Assumption

- Recall: the next word only on a single preceding word

I made **myself** a peanut butter sandwich.

I'll go and see her **myself**.

I'll go and see her **myself** a peanut butter sandwich.

I made myself a **peanut** butter sandwich.

The **peanut** is not a nut.

I made myself a **peanut** is not a nut.

Extending the Markov Model

- Instead of depending only on one past state, depend on two

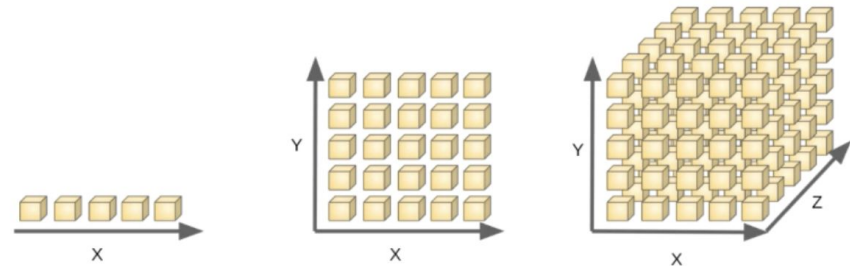
First Order Markov : $p(s_t \mid s_{t-1}, s_{t-2}, \dots) = p(s_t \mid s_{t-1})$

Second Order Markov : $p(s_t \mid s_{t-1}, s_{t-2}, \dots) = p(s_t \mid s_{t-1}, s_{t-2})$

2nd-Order Markov Model Implementation

$$A_{ijk} = p(s_t = k \mid s_{t-1} = j, s_{t-2} = i)$$

- This is a 3-D array
- Its shape is $M \times M \times M = M^3$
- 3rd-order: M^4 , 4th-order: M^5 , etc. This grows exponentially!



The Full Model

$$\pi_i = p(s_1 = i)$$

$$A^{(1)}_{ij} = p(s_2 = j \mid s_1 = i)$$

$$A^{(2)}_{ijk} = p(s_t = k \mid s_{t-1} = j, s_{t-2} = i)$$

"The quick brown fox jumps over the lazy dog."

Storing Word Probabilities in Dictionaries

- We will not use add-one smoothing, so many words will have zero probability (and they will simply not appear in the dictionary)
- Thus, although there will be V words in the vocabulary, the dictionary will not store V items

```
pi = {  
    "the": 0.2,  
    "a": 0.3,  
    "is": 0.1,  
    ...  
}
```

Why Use Dictionaries?

- Sparsity!
- We use add-one smoothing since many possible transition don't appear in the training corpus
- This gets worse as the model order gets larger
- i.e. Transition array will be **mostly zeros** (out of $V \times V \times V$ total elements)
 - It's more efficient to store only the transition which were seen!

First Order Transitions

- Recall: used only to model the 2nd word in each line

```
A1 = {  
    "the": {  
        "cat": 0.05,  
        "dog": 0.03,  
        "mouse": 0.01,  
        ...  
    },  
    "a": { ... },  
    ...  
}
```

First Order Transitions

- Recall: used only to model the 2nd word in each line

```
A1 = {  
  "the": {  
    "cat": 0.05,  
    "dog": 0.03,  
    "mouse": 0.01,  
    ...  
  },  
  "a": { ... },  
  ...  
}
```

$p(\text{cat} \mid \text{the}) = 0.05$

Exercise: how would you
store second-order
transitions?

Sampling from a Probability Dictionary

- How do we sample words from a dictionary with probabilities stored as dictionary values (with words as keys)?

```
probs = {  
    "the": 0.2,  
    "a": 0.3,  
    "is": 0.1,  
    ...  
}
```

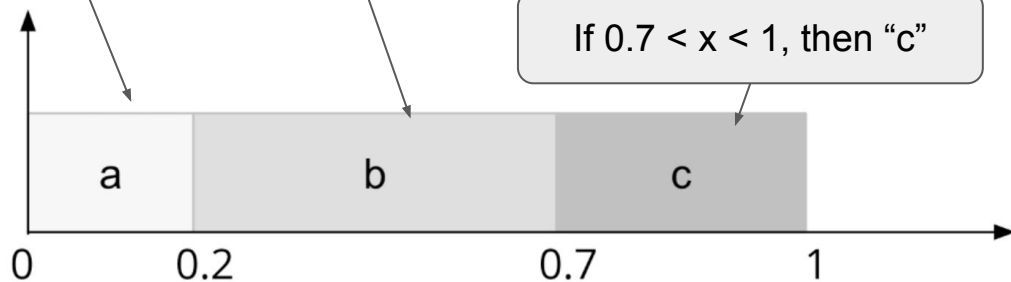
Example

Start by drawing a number $x \sim U(0, 1)$

If $0 < x < 0.2$, then "a"

If $0.2 < x < 0.7$, then "b"

If $0.7 < x < 1$, then "c"



- The **area** covered by each of these ranges is exactly the assigned probabilities (0.2, 0.5, 0.3)
- Notice: we must calculate the **cumulative sum**
0 + 0.2: first boundary
0.2 + 0.5 = 0.7: second boundary
0.7 + 0.3 = 1: third boundary

```
probs = {  
    "a": 0.2,  
    "b": 0.5,  
    "c": 0.3  
}
```

Pros & Cons of Markov Models

✓ Advantages:

- Simple & Efficient → Easy to implement and compute.
- Interpretable → Probabilities are clear and easy to analyze.
- Good for structured sequences → POS tagging, DNA sequence analysis.

✗ Limitations:

- Lacks long-term memory → Only considers the current state.
- Fails with ambiguous words → Struggles when context matters.
- Neural models (like Transformers) perform better → Capture long-range dependencies.